Smart Contract Assessment

HALBORN

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

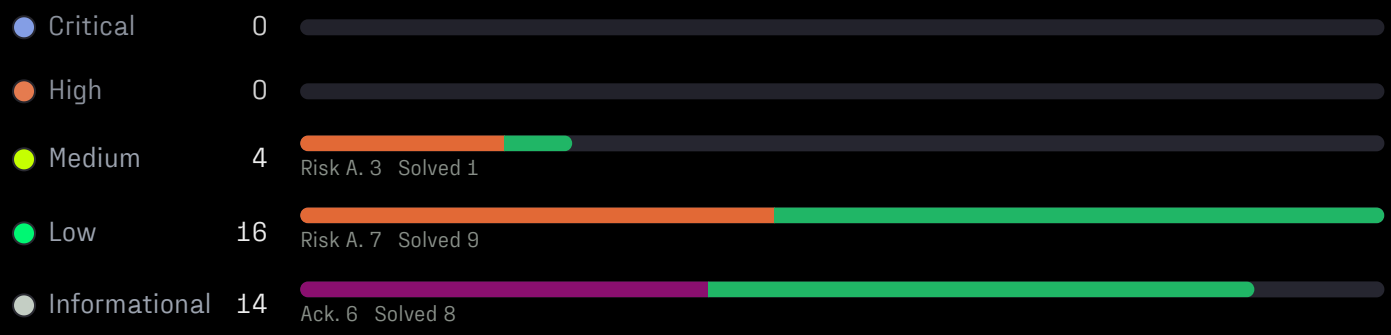
ABOUT THIS REPORT

ASSESSMENT TYPE
Assurance Assessment

DATE OF ENGAGEMENT
Jun 12, 2026 - Jun 27, 2026

SEVERITY BREAKDOWN

34	18	10	6
All Findings	Solved	Risk Accepted	Acknowledged



INTRODUCTION

`Client` engaged `Halborn` to perform a security assessment of their smart contracts from June 12th, 2026 to June 25th, 2026. The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The `Client` codebase in scope consists of a LINK liquid staking system built around a priority pool, staking pool, and withdrawal pool, supported by OperatorVCS and CommunityVCS strategies that route deposits into Chainlink staking vaults, alongside fund-flow and rebase controllers and an SDL-based rewards-sharing mechanism.

ASSESSMENT SUMMARY

`Halborn` was allocated 10 days for this engagement and assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, `Halborn` identified several areas for improvement to reduce the likelihood and impact of security risks, which were partially addressed by the `Client` team. The main recommendations were:

- `Replace each subtraction with a saturating subtraction that returns zero whenever staked principal exceeds the live per-vault limit, in VaultControllerStrategy.getMaxDeposits(), in the per-vault removed-room loop of OperatorVCS.getMaxDeposits(), and in FundFlowController._getTotalDepositRoom().`
- `In the per-iteration loop of FundFlowController._getVaultWithdrawalOrder(), exclude any vault from both the order and the counted total when it fails getPrincipalDeposits() != 0 && claimPeriodActive() && !isRemoved(), and add the same !isRemoved() check to the deposit ordering loop of _getVaultDepositOrder().`
- `Distribute rewards atomically at the moment they are received rather than allowing balances to accumulate in the controller before a permissionless distribution call.`

TEST APPROACH AND METHODOLOGY

`Halborn` conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.
The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts (`Foundry`).
- Fork testing against main networks (`Foundry`).
- Static security analysis of scoped contracts, and imported functions (`Slither`).

Risk Methodology

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A)	1

EXPLOITABILITY METRIC (m_e)	METRIC VALUE	NUMERICAL VALUE
	Specific (AO:S)	0.2
Attack Cost (AC)	Low (AC:L)	1
	Medium (AC:M)	0.67
	High (AC:H)	0.33
Attack Complexity (AX)	Low (AX:L)	1
	Medium (AX:M)	0.67
	High (AX:H)	0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (m_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N)	0
	Low (C:L)	0.25
	Medium (C:M)	0.5
	High (C:H)	0.75
	Critical (C:C)	1
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

Critical	High	Medium	Low	Informational
9 - 10	7 - 8.9	4.5 - 6.9	2 - 4.4	0 - 1.9

Scope

URLS

(a) URL: https://etherscan.io/address/0x855039b459cea56707a61e641110097623de1bc3

(b) Items in scope:

PriorityPool.sol Solidity

(a) URL: https://etherscan.io/address/0x88dd5c421f7b9fcdb83fd534bd83d22f8b80ea75

(b) Items in scope:

SDLPool.sol Solidity

(a) URL: https://etherscan.io/address/0x8753C00D1a94D04A01b931830011d882A3F8Cc72

(b) Items in scope:

RewardsPoolWSD.sol Solidity

(a) URL: https://etherscan.io/address/0x1711e93eec78ba83D38C26f0fF284eB478bdbec4

(b) Items in scope:

RebaseController.sol Solidity

(a) URL: https://etherscan.io/address/0x25fb2b9030f8c0002d018bb02b7d86b844baaf57

(b) Items in scope:

OperatorVCS.sol Solidity

(a) URL: https://etherscan.io/address/0xacbfc09a1c4966f11c969080374010536090a8e0

(b) Items in scope:

FundFlowController.sol Solidity

(a) URL: https://etherscan.io/address/0x2fe5a394f24b3dec644f982a9bb0bafe57308597

(b) Items in scope:

StakingPool.sol Solidity

(a) URL: https://etherscan.io/address/0x14024f4e7fef08cb19f8051456e39ee899990744

(b) Items in scope:

CommunityVCS.sol Solidity

(a) URL: https://etherscan.io/address/0x9e00fd7e4754cc0deca4d653c61190380318ed17

(b) Items in scope:

<> WithdrawalPool.sol Solidity

REMEDIATION COMMIT ID:

- 192617e8e13e4c9a32fc445467d695746004a1f0d
- 0b88fb7868fd5d947b9d66784f431b39676a7603
- 92ef7aabcbebbccbefa19ac85dad18cfbb9b60fb
- a3cb26ebc9509e13ee17e68ff4a2b767c796aace
- 244c0633137262e99390b4fdcd91126baac3f881
- 352d5484fe1ad9217743e014bb4f97e7495dd172
- ab875d7113de551fa78231636387f3b042d2892e
- fabbc3712e801030d8dca19e3df869b5a00784ed
- ec62bd5512b8b9766da3751e8a7abebc7b21af0a
- bd0f6fb545ad016cd9736bd1ae849022b9c9df13
- d8c4d4d1e5946f8752e47f881c9a04af7734c80b
- 58b8d7e418a31045c60265420de4580abcaa0de8
- 29e3a3893460b89fb960cad9dedc5977b8cb6a5f
- 018fb64e32563904fa8f83754354b3d6fc0ba75a
- 5d6513a7f4597345b6a2672cad694115cef2db82
- 656069b01d9799d405662397bd2dacd749cc98f0
- 4210a35e08a51ae8db6363639be4fa034f6ea9ae
- 274bd7f02186d866dbfdcdc61328da77b627a307

Out-of-Scope: New features/implementations after the remediation commit IDs.

Assessment Summary & Findings Overview

#	TITLE	SEVERITY	SCORE	STATUS
HAL-001	Deposit deployment and vault removal denial of service due to unguarded subtraction against the live Chainlink limit	Medium	5.0	RISK ACCEPTED 08/11/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-002	Overstated and understated routing due to missing vault eligibility checks in FundFlowController	Medium	5.0	SOLVED 07/14/2026
HAL-003	Theft of undistributed rewards due to non-time-weighted permissionless distribution	Medium	5.0	RISK ACCEPTED 08/11/2026
HAL-004	Corrupted vault grouping state on upgrade due to duplicated vault groups and a basis-point deposit index	Medium	5.0	RISK ACCEPTED 08/11/2026
HAL-005	Reward distribution blocked by a reverting fee receiver callback	Low	3.4	RISK ACCEPTED 08/13/2026
HAL-006	Rising gas cost on hot paths due to unbounded per-vault iteration	Low	3.4	RISK ACCEPTED 08/13/2026
HAL-007	Pool closure on any negative deposit change due to missing loss-proportionality threshold	Low	3.4	SOLVED 07/14/2026
HAL-008	Reward distribution blocked while total staked is zero, temporarily stranding reward tokens	Low	3.4	SOLVED 07/14/2026
HAL-009	Partial access control in updateVaultDeposits allows anyone to complete in-progress batches	Low	2.5	RISK ACCEPTED 08/11/2026
HAL-010	Combined fee rate up to 70% due to independently enforced strategy and pool fee caps	Low	2.5	RISK ACCEPTED 08/11/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-011	Reward distribution and vault removal halted by a single reverting vault in per-vault loops	Low	2.5	SOLVED 08/14/2026
HAL-012	Storage collision on future upgrade due to insufficient or missing storage gaps in base contracts	Low	2.5	RISK ACCEPTED 08/13/2026
HAL-013	Pool capacity invariant breach due to wrong deposit-room check in bypassQueue	Low	2.5	RISK ACCEPTED 08/13/2026
HAL-014	Reward distribution halt due to fee amounts converting to zero shares	Low	2.5	SOLVED 07/14/2026
HAL-015	Group deposit-room over-credited when reusing a removed ungrouped vault slot	Low	2.5	SOLVED 07/14/2026
HAL-016	Withdrawal dust shares discarded without crediting their value in finalization	Low	2.5	SOLVED 07/14/2026
HAL-017	Operator reward withdrawal denial of service due to unguarded subtraction in withdrawOperatorRewards	Low	2.5	SOLVED 07/14/2026
HAL-018	Permanent reward stranding due to round-down in _updateRewardPerToken	Low	2.5	SOLVED 07/14/2026
HAL-019	Deposit update stalled by permissionless vault deployment during an in-progress batch	Low	2.5	SOLVED 07/14/2026

#	TITLE	SEVERITY	SCORE	STATUS
HAL-020	Permanent loss of administrative control due to single-step ownership transfer	Low	2.0	RISK ACCEPTED 08/13/2026
HAL-021	Keeper transaction griefing due to exact-match vault deposit ordering check	Informational	1.7	ACKNOWLEDGED 08/14/2026
HAL-022	Opaque panics due to missing bounds and array-length validation	Informational	1.7	SOLVED 07/16/2026
HAL-023	Gas griefing through permissionless updateOperatorVaultGroupAccounting with caller-controlled array	Informational	1.7	ACKNOWLEDGED 08/14/2026
HAL-024	Withdrawal denial of service due to unguarded subtraction in getMinDeposits after slashing	Informational	1.7	SOLVED 07/16/2026
HAL-025	Unbounded iteration in getLockIdsByOwner can revert as the global lock counter grows	Informational	1.7	ACKNOWLEDGED 08/14/2026
HAL-026	Core ERC20 invariant broken because the dust guard hides small balances from balanceOf	Informational	0.8	ACKNOWLEDGED 08/11/2026
HAL-027	Transfer event reports nominal amount instead of the actual transferred value	Informational	—	SOLVED 07/16/2026
HAL-028	Checks-effects-interactions violation in removeVault during vault exit	Informational	—	ACKNOWLEDGED 08/14/2026
HAL-029	Loss of revert context due to discarded	Informational	—	SOLVED 08/16/2026

#	TITLE	SEVERITY	SCORE	STATUS
	delegatecall return data in deposit and withdraw			
HAL-030	Missing events on state-changing administrative functions	Informational	—	SOLVED 08/16/2026
HAL-031	Missing zero-address validation on critical immutable addresses	Informational	—	SOLVED 07/16/2026
HAL-032	Withdrawal can transfer more than the requested amount when avoiding a sub-minimum vault remainder	Informational	—	ACKNOWLEDGED 08/14/2026
HAL-033	Non-empty placeholder data passed to token-transfer callbacks	Informational	—	SOLVED 07/17/2026
HAL-034	Batch token distribution reverts entirely when any reward pool has a zero balance	Informational	—	SOLVED 07/17/2026

Findings & Tech Details

HAL-001 RISK ACCEPTED

Deposit deployment and vault removal denial of service due to unguarded subtraction against the live Chainlink limit

• Medium · 5.0 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:H/I:N/D:N/Y:N

DESCRIPTION

The `VaultControllerStrategy.getMaxDeposits()` function and the `FundFlowController._getTotalDepositRoom()` function compute available vault capacity by subtracting principal already staked in the Chainlink staking contract from a per-vault maximum read live from that same contract through `getStakerLimits()`. This capacity math sits on the deposit-deployment path and on the operator-removal path, so it must never revert under normal operation.

However, these subtractions have no lower-bound guard. In `getMaxDeposits()` the minuend is `vaults.length * maxDeposits` and in `_getTotalDepositRoom()` it is `_vaultMaxDeposits - principalDeposits` per vault, while the subtrahend is principal staked when an earlier, higher limit was in force. The same unguarded pattern exists in the per-vault removed-room loop of `OperatorVCS.getMaxDeposits()`. If the Chainlink staking contract lowers its per-vault staker limit after vaults are filled to the previous limit, the live limit becomes smaller than the stored principal and the subtraction underflows, reverting under Solidity 0.8 checked arithmetic.

The revert in `getMaxDeposits()` propagates through `StakingPool.canDeposit()` and the priority pool deposit-deployment path, blocking the deployment of queued and unused tokens into strategies, and blocking direct deposits when the queue is empty. The revert in `_getTotalDepositRoom()` blocks the permissionless `updateOperatorVaultGroupAccounting()` and, through it, `OperatorVCS.queueVaultRemoval()`, so a removed or slashed operator cannot be removed. Both conditions persist until the Chainlink limit is restored or enough principal is withdrawn.

For example:

1. Vaults are filled to the current Chainlink per-vault staker limit, so stored principal equals that limit for one or more vaults.

2. The Chainlink staking contract lowers its per-vault staker limit.

3. The deposit-deployment path calls `StakingPool.canDeposit()`, which calls `getMaxDeposits()` and reverts on the underflow, halting deployment of queued tokens.

4. Any account calls `updateOperatorVaultGroupAccounting()` or queues an operator for removal, routing into `_getTotalDepositRoom()` and reverting on the same underflow, blocking operator-vault removal.

RECOMMENDATION

Replace each subtraction with a saturating subtraction that returns zero whenever staked principal exceeds the live per-vault limit, in `VaultControllerStrategy.getMaxDeposits()`, in the per-vault removed-room loop of `OperatorVCS.getMaxDeposits()`, and in `FundFlowController._getTotalDepositRoom()`.

Additionally, source the per-vault limit used for deposit-room accounting from the strategy's stored `vaultMaxDeposits`, which only ever increases, so a transient Chainlink limit decrease cannot break the accounting.

REMEDIATION COMMENT

RISK ACCEPTED: The `stake.link` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> The arithmetic is as described, but the trigger cannot occur. The finding assumes Chainlink can lower maxPrincipalPerStaker below staked principal. Chainlink's StakingPoolBase._setConfig reverts on any decrease to maxPrincipalPerStaker, so getStakerLimits() is monotonically non-decreasing and the subtractions cannot underflow. The reachable sibling, a stale totalUnbonded in getMinDeposits after a slash, is a real case and is fixed under HAL-24.

HAL-002 SOLVED

Overstated and understated routing due to missing vault eligibility checks in FundFlowController

Medium · 5.0 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N

DESCRIPTION

The `FundFlowController` contract builds the ordered vault list and aggregate amount that `getWithdrawalData()` and `getDepositData()` use to split a withdrawal or deposit between the OperatorVCS and CommunityVCS strategies. For the split to be correct, the controller must count a vault's capacity only when the strategy will actually act on that vault, because the on-chain consumer applies strict eligibility checks before touching any vault.

However, the controller omits eligibility checks that its consumers enforce. In the withdrawal path, `FundFlowController._getVaultWithdrawalOrder()` adds the continuation vault's full `getPrincipalDeposits()` without checking `claimPeriodActive()` or `isRemoved()`, and its per-iteration loop checks `claimPeriodActive()` but omits `isRemoved()`, whereas the consumer `VaultDepositController.withdraw()` acts only when `deposits != 0 && claimPeriodActive() && !isRemoved()`. In the deposit path, `FundFlowController._getVaultDepositOrder()` adds room without checking `isRemoved()`, whereas the consumer excludes removed vaults.

Because routing sends the remainder to the second strategy only when the first cannot cover the request, an overstated total causes too little to be routed to the other strategy, so a withdrawal is filled partially or reverts with `InsufficientTokensUnbonded`, and a deposit leaves part unallocated.

For example:

1. A vault holds principal but is ineligible, namely a continuation vault re-bonded so its claim period is inactive, or a vault flagged as removed.
2. A user requests a withdrawal, so `_getVaultWithdrawalOrder()` counts that vault's principal into the community total, making the community strategy appear able to cover the request.
3. Less than the required amount is routed to the OperatorVCS, so `VaultDepositController.withdraw()` skips the ineligible vault and withdraws less than requested or reverts.

RECOMMENDATION

In the per-iteration loop of `FundFlowController._getVaultWithdrawalOrder()`, exclude any vault from both the order and the counted total when it fails `getPrincipalDeposits() != 0 && claimPeriodActive() && !isRemoved()`, and add the same `!isRemoved()` check to the deposit ordering loop of `_getVaultDepositOrder()`.

For the continuation vault at position zero, still keep it in the order to preserve the index pointer that the consumer requires, but exclude its principal or room from the counted total when it fails the predicate. This avoids the guaranteed `InvalidVaultIds` revert that would occur if the mandatory first vault were dropped from the array.

REMEDIATION COMMENT

Addressed in: 192617e8e13e4c9a32fc445467d6957460Oa1f0d

SOLVED: The `stake.link` **team** solved this finding in the specified commit by adding the missing `isRemoved()` / `claimPeriodActive()` eligibility checks to both the deposit and withdrawal ordering routines so the counted totals match what the on-chain consumer will act on, stating:

> Added the missing isRemoved() / claimPeriodActive() eligibility checks to FundFlowController deposit and withdrawal ordering, so routing totals match what the on-chain consumer acts on.

HAL-003 RISK ACCEPTED

Theft of undistributed rewards due to non-time-weighted permissionless distribution

Medium · 5.0 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:M/Y:N

DESCRIPTION

The `RewardsPool.distributeRewards()` function, together with `RewardsPool._updateRewardPerToken()`, credits newly received reward tokens to all current stakers by advancing `rewardPerToken` by `reward * 1e18 / totalStaked`, where `totalStaked` is the SDL pool's total effective balance. The `RewardsPoolController.distributeToken()` function that triggers distribution and `RewardsPool.distributeRewards()` itself are both permissionless, and reward tokens routinely accumulate in the controller before anyone triggers a distribution.

However, reward accrual is not time-weighted, so a staker is credited a full pro-rata slice of a distribution simply by holding effective balance at the instant `distributeRewards()` runs, regardless of how long they were staked. Any account can lock SDL with a `lockingDuration` of zero through `SDLPool.onTokenTransfer()`, which calls `_createLock()` with `startTime` set to zero, and `SDLPool.withdraw()` skips the unlock-period check entirely when `startTime` is zero. The attacker can therefore enter and exit within a single block.

This lets any account capture rewards that accrued before it staked, diluting the rewards that belong to long-term stakers. In one atomic transaction the attacker locks a large zero-duration SDL amount, triggers the distribution, withdraws the just-credited reward slice, and withdraws the SDL principal, contributing nothing during the accrual period while extracting a share of it.

For example:

1. Reward tokens accumulate in the SDL pool controller but `distributeRewards()` has not yet been called.
2. The attacker calls `sdlToken.transferAndCall` with a locking duration of zero, increasing `effectiveBalances` and `totalEffectiveBalance`, while the `updateRewards` modifier snapshots their `userRewardPerTokenPaid` at the current `rewardPerToken`.
3. The attacker calls `distributeToken(rewardToken)`, raising `rewardPerToken` using the inflated `totalStaked` that now includes the attacker.
4. The attacker's `withdrawableRewards` now includes a large pro-rata slice of the just-distributed batch even though they were not staked while it accrued.
5. The attacker withdraws the rewards through `withdrawRewards()` and immediately withdraws the zero-duration SDL principal, all in the same transaction.

RECOMMENDATION

Distribute rewards atomically at the moment they are received rather than allowing balances to accumulate in the controller before a permissionless distribution call. Alternatively, introduce time-weighted accounting so that an account's reward share reflects its staking duration over the accrual period.

Additionally, consider making zero-duration locks ineligible for rewards distributed in the same block in which they are created.

REMEDIATION COMMENT

RISK ACCEPTED: The `stake.link` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> The exploit requires reward tokens to sit undistributed in the controller so an attacker can lock a zero-duration SDL amount, trigger the pending distribution, and capture a slice in one transaction. That window does not exist: rewards are delivered via ERC677 transferAndCall, whose onTokenTransfer immediately calls distributeToken in the same transaction, so distribution is atomic with receipt. The residual is a theoretical raw-transfer path, which is not the reward-delivery mechanism the system uses; we accept that rather than add time-weighting to a live token.

HAL-004 RISK ACCEPTED

Corrupted vault grouping state on upgrade due to duplicated vault groups and a basis-point deposit index

Medium · 5.0 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:M/D:N/Y:N

DESCRIPTION

The `OperatorVCS.initialize()` function (declared `reinitializer(4)`) and the `CommunityVCS.initialize()` function (declared `reinitializer(2)`) both contain an upgrade re-initialization branch taken when the strategy token is already set on an already-deployed proxy. That branch rewrites the strategy's core vault accounting and re-seeds the `vaultGroups` array, both of which deposit routing in `VaultDepositController._depositToVaults()` depends on. The `vaultGroups` array partitions operator vaults into groups by index, and `globalVaultState.depositIndex` defines the boundary between grouped and ungrouped vaults.

However, the trailing loop `for (uint64 i = 0; i < 5; ++i) vaultGroups.push(VaultGroup(i, 0))` runs unconditionally in both the fresh-initialization branch and the upgrade branch, with no guard such as `if (vaultGroups.length == 0)`. An upgraded proxy that already holds five populated `vaultGroups` entries therefore gets five more appended, leaving ten total entries while `numVaultGroups` stays at five. The trailing entries five through nine become stale, never-rotated group state that no longer reflects the live grouping. Both the OperatorVCS and CommunityVCS paths perform this unconditional push.

Separately, the same upgrade branch sets `globalVaultState = GlobalVaultState(5, 0, 0, uint64(maxDepositSizeBP + 1))`, assigning the basis-point quantity `maxDepositSizeBP + 1` into the `depositIndex` field. The value read is the previously-stored `maxDepositSizeBP`, because the assignment `maxDepositSizeBP = _maxDepositSizeBP` executes on the line after the struct write. The `maxDepositSizeBP` value is a basis-point quantity bounded to at most 10000 and bears no relationship to the number of vaults the strategy controls, so writing it into a vault-index field can set the grouped/ungrouped boundary to a value unrelated to the actual vault set. When `depositIndex` exceeds `vaults.length`, the ungrouped fill loop in `VaultDepositController._depositToVaults()` never executes, deposits never reach the ungrouped vaults, and per-group deposit-room accounting in `vaultGroups[i].totalDepositRoom` is consumed against an incorrect boundary.

For example:

1. A strategy is already deployed and initialized, with vaults and five vault groups in active use.
2. The proxy is upgraded and `initialize()` runs again under its reinitializer, taking the upgrade branch because the token is already set.

3. The unconditional push loop appends five more `vaultGroups` entries, so the array holds ten entries while `numVaultGroups` remains five, leaving stale group state.
4. `globalVaultState.depositIndex` is set to the previously-stored `maxDepositSizeBP + 1`, a basis-point value unrelated to the real grouped-vault count. If that value exceeds `vaults.length`, subsequent deposits route through `_depositToVaults()` against an incorrect boundary, the ungrouped fill loop never executes, and group deposit-room accounting becomes inconsistent.

RECOMMENDATION

Guard the `vaultGroups` initialization loop so that groups are only pushed when `vaultGroups` is empty, preventing duplicate group entries on the reinitialization path in both `OperatorVCS` and `CommunityVCS`.

Additionally, set `globalVaultState.depositIndex` on the upgrade path to the actual grouped-vault boundary intended for the migrated state rather than to `maxDepositSizeBP + 1`, and validate that the written value is a real vault index bounded by `vaults.length`. Confirm against the intended migration that the boundary written matches the real grouped-vault count.

REMEDIATION COMMENT

RISK ACCEPTED: The `stake.link` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> The report reads GlobalVaultState(5, 0, 0, uint64(maxDepositSizeBP + 1)) as writing a basis-point value into the vault-index field. This misreads a deliberate storage-slot alias. The reinitializer(3) migration (commit 8d4c7dc) removed the old indexOfLastFullVault variable, so at the moment that line executes the slot the new code names maxDepositSizeBP still physically holds the migrated indexOfLastFullVault; the real basis-point value is assigned on the next line. indexOfLastFullVault + 1 was the old "next vault to fill" pointer, so maxDepositSizeBP + 1 correctly seeds depositIndex. Verified against live mainnet state (OperatorVCS proxy 0x4852...): depositIndex == 15 == getVaults().length, numVaultGroups == 5, vaultGroups.length == 5. The remaining "duplicate vaultGroups on re-run" concern is the generic hazard shared by every reinitializer-guarded initialize in the system, not a VCS-specific defect.

HAL-005 RISK ACCEPTED

Reward distribution blocked by a reverting fee receiver callback

Low · 3.4 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The `StakingPool._updateStrategyRewards()` function distributes protocol fees by minting liquid staking shares to the pool and transferring them to each configured fee receiver through `transferAndCallFrom()`. For receivers that are contracts, this invokes the ERC677 `contractFallback()`, which calls `onTokenTransfer()` on the receiver.

However, neither `_updateStrategyRewards()` nor `contractFallback()` isolates the receiver callback, so a single fee receiver whose `onTokenTransfer()` reverts causes the entire reward update to revert. This rolls back the `totalStaked` accrual and the fee share minting for that update, so no rewards are distributed to anyone. Reward accounting then remains stuck because the only way to remove the offending receiver, `updateFee()`, itself calls `_updateStrategyRewards()` and is therefore also blocked while the receiver reverts.

A buggy or maliciously reverting fee receiver halts all reward and fee accounting for every strategy and every staker. Fee receivers are configured by the owner, so realization depends on a configured receiver entering a reverting state.

RECOMMENDATION

Isolate each fee distribution call so that a failing receiver cannot revert the whole update, for example by wrapping the per-receiver transfer in a try/catch and emitting an event on failure. Do not simply leave the undelivered fee sitting in the contract: the final receiver is paid the pool's full remaining balance, so any fee left behind would be silently swept to whichever receiver is processed last.

Instead, either avoid minting the shares attributable to a skipped receiver, or record the undelivered amount to an explicit recovery accounting entry, and pay the final receiver its own computed fee amount rather than the full remaining balance so that a skipped fee is never redirected. Ensure the critical `totalStaked` accounting update is committed regardless of whether an individual receiver callback succeeds.

REMEDIATION COMMENT

RISK ACCEPTED: The `stake.link` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> Technically correct, but addFee / updateFee are onlyOwner, so an attacker can never register a receiver, and the live receiver set is small and protocol-owned. Managed operationally by

> controlling which receivers are registered. If untrusted receivers are ever introduced, we will isolate the per-receiver callback in try/catch while keeping the final-receiver sweep.

HAL-006 RISK ACCEPTED

Rising gas cost on hot paths due to unbounded per-vault iteration

Low · 3.4 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The Vault Controller Strategy contracts and the `FundFlowController` iterate over their full vault arrays on the reward-update, deposit, and vault-group-rotation paths, which are the routines that keep deposits, reward distribution, and withdrawals functioning. Because vaults are deployed in unbounded batches over time, any per-vault loop on one of these frequently used paths grows linearly in cost with the vault count.

However, none of these loops is bounded or batched. The `CommunityVCS.updateDeposits()` function falls back to `getDepositChange()` whenever `vaultsPerBatch` is zero, its default uninitialized value, and `getDepositChange()` iterates every vault calling `getTotalDeposits()` on each. The `OperatorVCS.getMaxDeposits()` function, which the staking pool calls on every deposit, iterates every vault and makes two external calls per vault, namely `isRemoved()` and `getPrincipalDeposits()`, whenever `vaultsToRemove` or `removedVaults` is non-empty, and `removedVaults` is only cleared when `OperatorVCS.addVault()` reuses a slot. The `OperatorVCS.updateDeposits()` function, reached through every reward update, fee change, and strategy removal, always iterates the full vault array making one `IOperatorVault.updateDeposits()` call per vault with no pagination. The `FundFlowController.updateVaultGroups()` function computes update data and unbonds the current group across both strategies, with the number of external calls growing with the combined vault count. The `OperatorVCS.queueVaultRemoval()` function compounds the cost with linear duplicate-detection scans over both removal arrays.

As the vault and operator counts grow, reward updates, deposits, and vault-group rotations become increasingly expensive and can approach the block gas limit, at which point reward distribution, deposits, and the rotation that withdrawals depend on can become unexecutable. No accounting is corrupted, since all values are read live from vault state.

RECOMMENDATION

Ensure `CommunityVCS.updateDeposits()` always uses its batched path by enforcing a non-zero `vaultsPerBatch` once the vault count is allowed to grow, and introduce an analogous batching mechanism in `OperatorVCS.updateDeposits()` so that vault iteration during reward accounting can be split across multiple transactions. Additionally, track the deposit room attributable to removed vaults in a dedicated storage variable updated when a vault is queued for removal and when a slot is reused, so `OperatorVCS.getMaxDeposits()` does not iterate every vault, and replace the linear duplicate-detection scans in `OperatorVCS.queueVaultRemoval()` with mapping-based lookups. Additionally, split the per-vault read of update data from the unbonding

step in `FundFlowController.updateVaultGroups()` and allow the current group to be unbonded in bounded batches, or maintain per-group deposit room and unbonded totals incrementally so the rotation does not require iterating every vault.

REMEDIATION COMMENT

RISK ACCEPTED: The `stake.link` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> The recommended batching is already live. CommunityVCS (377 vaults) runs updateVaultDeposits in vaultsPerBatch = 200 chunks, so the full-iteration fallback is not the live path. OperatorVCS is a small permissioned set (15 vaults), and updateVaultGroups iterates a single group. Optional hardening not applied: enforce vaultsPerBatch != 0 in the setter.

HAL-007 SOLVED

Pool closure on any negative deposit change due to missing loss-proportionality threshold

Low · 3.4 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The `RebaseController` contract is the component that closes the staking pool and initiates a security-pool claim when a strategy reports a loss, acting as slashing protection. The intended behavior is that only material losses escalate to closing the pool, while smaller losses are handled proportionally.

However, both `RebaseController.checkUpkeep()` and `RebaseController.performUpkeep()` treat any `getDepositChange() < 0` as a trigger, with no minimum loss threshold and no proportionality between loss magnitude and response. `checkUpkeep()` returns true for the first strategy with a negative deposit change, and `performUpkeep()` closes the pool and initiates a claim if the indicated strategy's deposit change is below zero. A negative deposit change arises from a genuine slashing event in the Chainlink staking contract, but even a slash of trivial magnitude escalates to a full pool shutdown. Because `performUpkeep()` is permissionless, anyone can trigger this escalation the moment a negative change exists. The owner can recover by calling `reopenPool()`, which re-syncs strategy accounting through `updateStrategyRewards()` so the deposit change returns to zero; a fresh escalation would require a new negative deposit change.

The consequence is that any slashing event, however small, forces the priority pool into CLOSED and initiates a security-pool claim, rather than being absorbed proportionally.

RECOMMENDATION

Introduce a configurable loss threshold and only trigger the closure-and-claim path in `performUpkeep()`, and the true return in `checkUpkeep()`, when the magnitude of the negative deposit change exceeds that threshold. Route sub-threshold negative changes through the normal `updateStrategyRewards` rebase flow so they are absorbed proportionally instead of closing the pool.

REMEDIATION COMMENT

Addressed in: 0b88fb7868fd5d947b9d66784f431b39676a7603

SOLVED: The `stake.link` **team** solved this finding in the specified commit by adding a configurable `lossThreshold` to `RebaseController` so the pool only closes when the loss magnitude exceeds it, stating:

> Added a configurable lossThreshold to RebaseController; the pool only closes when the loss magnitude exceeds it, so sub-threshold losses rebase normally.

HAL-008 SOLVED

Reward distribution blocked while total staked is zero, temporarily stranding reward tokens

Low · 3.4 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:M/I:N/D:N/Y:N

DESCRIPTION

The `RewardsPool.distributeRewards()` and `RewardsPoolWSD.distributeRewards()` functions account newly received reward tokens and advance the per-share accumulator. Both require `controller.totalStaked() > 0` and revert otherwise. Reward tokens can be present in the pool before distribution, because the controller transfers reward tokens to the pool and then calls `distributeRewards()`, and because any account can transfer reward tokens directly to the pool address.

However, if every staker unstakes so that `controller.totalStaked()` becomes zero while undistributed reward tokens are sitting in the pool, every call to `distributeRewards()` reverts on the `totalStaked > 0` requirement. The `totalStaked > 0` check is also a necessary division-by-zero guard inside `_updateRewardPerToken()`, so it cannot simply be removed. The reward tokens cannot be accounted or advanced into the accumulator until at least one account stakes again.

No staker's already-owed rewards are affected: reward withdrawals through `withdraw()` are computed from each account's `userRewards` and the existing `rewardPerToken`, independently of `totalStaked`. The impact is limited to the undistributed balance sitting in the pool, which stays unaccounted for the duration of the zero-stake window. The tokens are not lost: once staking resumes, the next `distributeRewards()` call picks up the full balance via `token.balanceOf(address(this)) - totalRewards`.

RECOMMENDATION

Handle the zero-staked case gracefully so the call does not revert. Return early when `controller.totalStaked()` is zero while retaining the undistributed balance, which the next `distributeRewards()` call will fold into its computation once `totalStaked` becomes positive again. Note that `RewardsPool` and `RewardsPoolWSD` do not currently expose any owner or admin role, so any privileged recovery path for the degenerate case where no account ever stakes again would require introducing such a role explicitly.

REMEDIATION COMMENT

Addressed in: 92ef7aabcbebbccbefa19ac85dad18cfbb9b60fb

SOLVED: The `stake.link` **team** solved this finding in the specified commit by making `distributeRewards` return early when `totalStaked == 0` instead of reverting, so the balance folds into the next distribution, stating:

> distributeRewards returns early when totalStaked == 0 instead of reverting; the balance folds into the next distribution.

HAL-009 RISK ACCEPTED

Partial access control in updateVaultDeposits allows anyone to complete in-progress batches

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `CommunityVCS.updateVaultDeposits()` function processes vault deposit accounting in batches. It is intended to be driven by a single privileged address, the `depositUpdater`, which starts a batch and advances it across multiple calls until every vault is processed. The accumulated total is later consumed by `updateDeposits()`, which is restricted to the staking pool and reverts while the batch is incomplete.

However, the access control check is written as `if (currentVaultIndex == 0 && msg.sender != depositUpdater) revert SenderNotAuthorized();`, so the `depositUpdater` restriction is enforced only at the start of a batch when `currentVaultIndex == 0`. Once the `depositUpdater` has started a batch and `currentVaultIndex` is non-zero, the compound condition is false for every caller, allowing any account to advance the batch to completion.

An unauthorized account can therefore drive an in-progress batch to completion, setting `currentVaultIndex` to `vaults.length` and controlling the timing and sequencing of the maintenance routine. This makes the strategy eligible for `updateDeposits()` earlier than the `depositUpdater` intended. Deposits and withdrawals remain blocked throughout, because `currentVaultIndex` stays non-zero until `updateDeposits()` resets it to zero. The accumulated values are objective on-chain reads of `vaults[i].getTotalDeposits()`, so no false accounting data can be injected.

RECOMMENDATION

Enforce the `depositUpdater` authorization on every call rather than only at the start of a batch by requiring `msg.sender == depositUpdater` unconditionally in `updateVaultDeposits()`.

REMEDIATION COMMENT

RISK ACCEPTED: The `stake.link` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> Only starting a batch is privileged; completion is permissionless by design. While a batch is in progress, deposits, withdrawals, and claims are blocked, so permissionless completion is a liveness escape hatch if the depositUpdater stalls. No accounting can be corrupted: the caller only accumulates objective on-chain reads in vault order, and updateDeposits stays onlyStakingPool.

HAL-010 RISK ACCEPTED

Combined fee rate up to 70% due to independently enforced strategy and pool fee caps

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The protocol charges fees on staking rewards at two independent levels. The `StakingPool` enforces a pool-level fee cap by requiring `_totalFeesBasisPoints() <= 4000` in `addFee()`, `updateFee()`, and `initialize()`, limiting pool fees to 40% of rewards. The `VaultControllerStrategy` enforces a separate strategy-level cap by reverting with `FeesTooLarge` when `_totalFeesBasisPoints() > 3000`, limiting strategy fees to 30% of rewards. Both sets of fees are configured by the owner.

However, these two caps are evaluated in separate contracts with no cross-contract validation, and both fee levels are applied to the same gross reward figure rather than being layered. A strategy reports its rewards to the pool as a positive deposit change. Inside the strategy, `VaultControllerStrategy.updateDeposits()` computes each strategy fee as a basis-point share of that gross deposit change, and the deposit change the strategy returns to the pool is not reduced by the strategy fee. `StakingPool._updateStrategyRewards()` then computes each pool fee as a basis-point share of that same gross deposit change. Because both levels take a cut of the identical gross reward amount and nothing relates the two caps, the owner can set strategy fees at 30% and pool fees at 40% simultaneously, and each individual cap check still passes.

The result is that stakers can be charged a combined effective fee of up to 70% of their rewards, far larger than either single cap suggests, with no on-chain check flagging the combined rate. The configuration is owner-controlled, so realization depends on the owner setting both caps to their maximums.

RECOMMENDATION

Enforce a combined fee ceiling that accounts for both levels rather than capping each in isolation. Have `StakingPool.addFee()` and `StakingPool.updateFee()` query each registered strategy's total fee basis points and include them in the cap check, or lower the individual caps so that their sum cannot exceed the intended maximum effective fee. At a minimum, emit an event carrying the combined effective fee rate whenever either level changes so that the aggregate can be monitored off-chain.

REMEDIATION COMMENT

RISK ACCEPTED: The `stake.link` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> Both caps are onlyOwner; reaching 70% requires the protocol multisig to deliberately set pool fees to 40% and strategy fees to 30% at once, a visible and self-defeating governance action, not an attacker or accidental path. A hard combined ceiling can be added later by summing pool and strategy fees against one limit.

HAL-011 SOLVED

Reward distribution and vault removal halted by a single reverting vault in per-vault loops

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

The `VaultControllerStrategy.updateVaultGroups()` and `OperatorVCS.updateDeposits()` functions iterate over every vault in a single loop and make an external call to each one with no isolation around the call. These loops drive vault group rotation and reward accounting, both of which are required for withdrawals and reward distribution to keep functioning.

However, neither loop tolerates a single failing vault. `VaultControllerStrategy.updateVaultGroups()` loops over the supplied vaults calling `vault.unbond()` with no error handling, so if any one vault is in a state that the Chainlink staking contract rejects for unbonding, the entire call reverts. Because `FundFlowController.updateVaultGroups()` invokes both strategies in the same transaction, the failure of one vault blocks group rotation for the whole protocol and the withdrawals that depend on it. Separately, `OperatorVCS.updateDeposits()` loops over every vault calling `IOperatorVault.updateDeposits()` with no isolation, so a single vault that reverts, for example after a slashing event leaves its internal accounting in a reverting state, blocks all reward distribution.

This compounds with removal. `OperatorVCS.removeVault()` calls the strategy reward update first, which iterates every vault including the broken one, so the broken vault cannot be removed because removing it requires the very iteration that the broken vault makes revert. The result is that one vault stuck in a reverting state halts vault group rotation, halts all reward and fee accounting, and cannot be removed through the normal path until it is repaired or an alternative route exists.

RECOMMENDATION

Provide a vault-removal path that does not require a preceding full reward update over all vaults, so a broken vault can be exited without first iterating it. For the per-vault loops, isolate the external call to each vault so a single failure cannot revert the whole operation, but do not fall back to last-known deposit or principal values. `OperatorVCS.updateDeposits()` does not persist per-vault deposit and principal amounts, so silently excluding a skipped vault from the accumulated vault deposits would make the computed deposit change negative by the size of the skipped vault, fabricating a loss that can trigger the pool-closure path in the rebase controller. Instead, route a reverting vault into an explicit quarantine state and reconcile `totalDeposits`, `totalPrincipalDeposits`, and the per-group unbonded totals deliberately as part of that quarantine, so a skipped vault is removed from the accounting in a controlled way rather than treated as a sudden loss.

REMEDIATION COMMENT

Addressed in: a3cb26ebc9509e13ee17e68ff4a2b767c796aace

SOLVED: The `stake.link` **team** solved this finding in the specified commit by refactoring the removal body into `_removeVault` and adding an owner-gated `removeVaultSkipRewardUpdate` that removes a queued vault without the preceding full reward update, while deliberately not adding per-vault call isolation, stating:

> The finding is accurate: the per-vault loops have no isolation, and removeVault ran the full reward update first, so a single reverting vault could not be removed (self-locking). Fixed: we refactored the removal body into _removeVault and added an owner-gated removeVaultSkipRewardUpdate that removes a queued vault without the preceding reward update, breaking the self-lock. Principal is settled from exitVault's returned amounts and accounting self-heals on the next updateStrategyRewards (verified by test). The skip path forfeits the departing operator's reward split for the current window, so it is owner-only and reserved for when normal removal is blocked. Deliberately not applied: per-vault try/catch isolation. As the report itself warns, silently skipping a reverting vault would fabricate a large negative depositChange and trigger the pool-closure path (HAL-07), turning a transient halt into a false slashing event. A correct version needs an explicit vault-quarantine mechanism, a feature-sized change best scoped on its own. Reachability is low: operator vaults run a protocol-controlled implementation that does not revert in these paths.

HAL-012 RISK ACCEPTED

Storage collision on future upgrade due to insufficient or missing storage gaps in base contracts

Low · 2.5 AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:H/D:N/Y:N

DESCRIPTION

The upgradeable base contracts in the strategy and staking inheritance chains are expected to reserve a trailing storage gap so that new state variables can be added in a future upgrade without shifting the storage slots used by the child contracts that inherit from them. The `VaultControllerStrategy`, `Strategy`, `StakingRewardsPool`, and `RewardsPoolController` base contracts either under-reserve this space or omit it entirely.

However, these four bases do not preserve a stable child layout across upgrades. The `VaultControllerStrategy` contract, the shared base of `CommunityVCS` and `OperatorVCS`, reserves only three gap slots, and two variables, `vaultDepositController` and `delegateRegistry`, have already been appended after `vaultMaxDeposits`, so a future upgrade that adds four or more base variables would overflow into the first child slots and overwrite vault deployment gating in `CommunityVCS` and operator reward accounting in `OperatorVCS`. The `Strategy` contract, the root of the same chain, declares no gap at all. The `StakingRewardsPool` contract, the base of `StakingPool`, declares no gap. The `RewardsPoolController` contract, the base of `SDLPool`, declares no gap.

There is no impact on the currently deployed state. The risk materializes only at a future upgrade that adds state to one of these base contracts, at which point deposit, withdrawal, reward, and lock accounting would read from incorrect storage slots and behave incorrectly or corrupt live balances.

RECOMMENDATION

For `VaultControllerStrategy`, where a three-slot gap is already deployed, do not enlarge the existing gap, because increasing its size would itself shift every child storage slot and cause the exact collision described here. Instead, treat the existing three slots as the fixed reserve: when a new base variable is later added, declare it in place of one of the gap slots and reduce the gap array by the same amount, so the total slot count and the child layout stay unchanged with no migration required. If more than three additional base variables are ever needed, that expansion requires a coordinated migration, the same as for the gapless bases below.

For `Strategy`, `StakingRewardsPool`, and `RewardsPoolController`, which declare no gap at all, note that these contracts are already deployed with child variables packed immediately after the base variables. Appending any new gap to one of these bases shifts every child storage slot and therefore cannot be done as a transparent change: it requires a coordinated migration upgrade that

re-aligns the child layout. Introduce the gap only as part of such a migration, and do not assume that adding the gap alone preserves the current layout for these three contracts.

REMEDIATION COMMENT

RISK ACCEPTED: The `stake.link` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> No effect on current state; children are packed correctly. The risk is purely prospective on a future upgrade adding base state. Naively adding gaps to the three gapless bases would itself shift every child slot on deployed proxies, so this is handled as a documented upgrade-time discipline (shrink the existing gap in place; introduce new gaps only within a coordinated migration).

HAL-013 RISK ACCEPTED

Pool capacity invariant breach due to wrong deposit-room check in bypassQueue

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `PriorityPool.bypassQueue()` function lets the queue bypass controller stake asset tokens directly into the staking pool, skipping the deposit queue. Before staking, it checks available capacity by comparing the requested amount against `StakingPool.getStrategyDepositRoom()`. The standard deposit path instead checks capacity against `StakingPool.canDeposit()`, and the two functions return different values.

However, `StakingPool.getStrategyDepositRoom()` sums each strategy's `canDeposit()` (the per-strategy maximum minus that strategy's deposits) and excludes unused asset tokens that are sitting in the pool but not yet deployed into strategies, whereas `StakingPool.canDeposit()` returns `getMaxDeposits()` minus `totalStaked`, and `totalStaked` already counts those unused deposits. As a result, `getStrategyDepositRoom()` equals `canDeposit()` plus the unused deposits. When the pool is at its maximum capacity (`canDeposit()` returns zero) but still holds unused deposits, `getStrategyDepositRoom()` remains positive, so `bypassQueue()` accepts a deposit that the regular path would reject.

When the queue bypass controller deposits up to the unused amount through `bypassQueue()`, `StakingPool.deposit()` mints liquid staking tokens and increments `totalStaked`, pushing `totalStaked` above `getMaxDeposits()` and minting stLINK beyond the redeemable capacity of the underlying strategies. The `InvalidDeposit` guard in `StakingPool.deposit()` does not trigger because the unused balance is redeployed into strategies, leaving the ending token balance equal to the starting balance. The breach is not self-correcting.

RECOMMENDATION

Change `bypassQueue()` to gate its capacity check on `StakingPool.canDeposit()` rather than `StakingPool.getStrategyDepositRoom()`, so that bypass deposits respect the same maximum-capacity limit enforced on the regular deposit path and `totalStaked` cannot exceed `getMaxDeposits()`.

REMEDIATION COMMENT

RISK ACCEPTED: The `stake.link` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> bypassQueue gates on getStrategyDepositRoom() rather than canDeposit() by design. Its only caller is LINKMigrator, moving LINK a user just unstaked from Chainlink's community pool 1:1 into the staking pool; that LINK is matched to room that genuinely exists on the strategy side, which

> is what getStrategyDepositRoom measures. Gating on canDeposit would spuriously reject legitimate migrations whenever the pool holds undeployed deposits. The transient totalStaked > getMaxDeposits is the intended migrator condition, backed by real freed Chainlink capacity, and the path is permissioned (bypass controller to migrator only).

HAL-014 SOLVED

Reward distribution halt due to fee amounts converting to zero shares

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `StakingPool._updateStrategyRewards()` function distributes protocol fees by minting liquid staking shares to the pool and then transferring stLINK out to each configured fee receiver through `transferAndCallFrom()`. Each non-final receiver is paid its precomputed amount, computed as `totalRewards` times `basisPoints` divided by 10000, while the final receiver is paid the pool's full remaining balance so any rounding remainder is swept to it.

However, `transferAndCallFrom()` routes into `StakingRewardsPool._transfer()`, which converts the token amount to shares with `getSharesByStake()` and reverts with "Transfer amount too small" whenever the derived share amount is zero. Two conditions produce a zero share amount in a real distribution. For a non-final receiver, when the pool has accrued positive rebases so the share price exceeds one token, a small fee amount, such as a receiver with a small basis-point share or a low-reward epoch, rounds down to zero shares. For the final receiver, the swept balance is read through `StakingRewardsPool.balanceOf()`, whose dust guard returns zero whenever the token-equivalent of the pool's fee shares is below one hundred, so the final transfer passes a zero value into `_transfer()`. In either case the per-receiver transfer reverts and the entire reward update reverts.

This rolls back the `totalStaked` accrual and the fee share minting for the epoch, so no rewards are distributed to anyone. It also blocks `StakingPool.addFee()`, `StakingPool.updateFee()`, and `StakingPool.removeStrategy()`, since all of them call `_updateStrategyRewards()`. Reward accounting stays stuck until rewards grow large enough that every receiver's fee amount converts to at least one share and the final swept balance rises above the dust threshold, so the effect is a transient halt of reward updates during low-reward epochs.

RECOMMENDATION

In `StakingPool._updateStrategyRewards()`, skip any per-receiver transfer whose computed fee amount converts to zero shares rather than letting it revert, and skip the final sweep when the pool's balance is zero, accumulating any skipped dust into a later distribution so it is not lost. Alternatively, transfer fee shares directly using a share-denominated transfer that does not pass through the token-amount rounding, the "Transfer amount too small" guard, and the `balanceOf()` dust guard.

REMEDIATION COMMENT

Addressed in: 244c0633137262e99390b4fdcd91126baac3f881

SOLVED: The `stake.link` **team** solved this finding in the specified commit by skipping any per-receiver fee transfer that converts to zero shares and skipping the final sweep at zero balance, stating:

> Skip any per-receiver fee transfer that converts to zero shares, and skip the final sweep at zero balance, so a low-value epoch cannot revert the reward update.

HAL-015 SOLVED

Group deposit-room over-credited when reusing a removed ungrouped vault slot

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `OperatorVCS.addVault()` function deploys a new operator vault and, when prior vaults have been removed, reuses a freed slot by placing the new vault at the index of a previously removed vault. Operator vaults are partitioned into vault groups by their index (`index % numVaultGroups`), and each group tracks the deposit capacity available to it through `vaultGroups[group].totalDepositRoom`, which the deposit routing in `VaultControllerStrategy` and `VaultDepositController` relies on to decide how much can be deposited into each group.

However, when reusing a slot, `OperatorVCS.addVault()` unconditionally credits `vaultGroups[group].totalDepositRoom += vaultMaxDeposits` for the reused index without checking whether that index belongs to an active vault group. Group deposit room is only meaningful for grouped vaults, namely those whose index is below `globalVaultState.depositIndex`; vaults at or above `depositIndex` are ungrouped and never contributed deposit room to any group.

This case is reachable. The `OperatorVCS.queueVaultRemoval()` function only requires that the vault report `isRemoved()` and does not restrict removal to grouped vaults, so an ungrouped vault (index at or above `depositIndex`) can be queued. The `OperatorVCS.removeVault()` function is public and permissionless, so any account can drive that ungrouped index into the `removedVaults` array. A subsequent `addVault()` then reuses the ungrouped index and credits a group's `totalDepositRoom` by `vaultMaxDeposits` for a vault that was never an active member of that group. As a result `vaultGroups[group].totalDepositRoom` is inflated by `vaultMaxDeposits` relative to the group's real capacity, so deposits can be routed assuming capacity that does not align with the actual in-group vault capacity. No funds are stolen, and the value is partially self-healing on later full recomputations.

RECOMMENDATION

Adjust the group deposit room only when the reused index actually belongs to an active group, by guarding the credit with a check that the index is below `globalVaultState.depositIndex` before applying `vaultGroups[group].totalDepositRoom += vaultMaxDeposits`. For reused ungrouped indexes, skip the group-room adjustment entirely.

REMEDIATION COMMENT

Addressed in: 352d5484fe1ad9217743e014bb4f97e7495dd172

SOLVED: The `stake.link` **team** solved this finding in the specified commit by guarding the group deposit-room credit so reusing a removed ungrouped vault slot no longer over-credits a group, stating:

> Guard the group deposit-room credit so reusing a removed ungrouped vault slot no longer overcredits a group.

HAL-016 SOLVED

Withdrawal dust shares discarded without crediting their value in finalization

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

The `WithdrawalPool._finalizeWithdrawals()` function finalizes queued LST withdrawals from the front of a FIFO queue as liquidity becomes available. When the finalized amount only partially covers the boundary withdrawal at index `i`, the function reduces `queuedWithdrawals[i].sharesRemaining` and credits `partiallyWithdrawableAmount` with the stake value of the shares just finalized.

However, the subsequent dust-handling block runs only when `sharesRemaining` is still positive after the partial finalization, meaning those leftover shares were never covered by the finalized amount and have no backing tokens in the pool. The block subtracts those sub-100 shares from `totalQueuedShareWithdrawals` and deletes `sharesRemaining`, so the owner's withdrawal is left with fewer shares than they queued and the corresponding value is detached from any future finalization.

Any account that ends up as the partially finalized boundary withdrawal with fewer than 100 remaining shares loses the value of those shares. The loss is bounded to under 100 share-equivalent of underlying token per affected withdrawal.

RECOMMENDATION

Do not delete and credit the dust shares, because they have no backing tokens at the time of the dust block and crediting their token value would make the pool pay more than it received, reducing the backing available to other queued withdrawals. Instead, leave the dust shares queued by keeping them in `queuedWithdrawals[i].sharesRemaining` and in `totalQueuedShareWithdrawals`, so their value is finalized against real liquidity in a later batch.

REMEDIATION COMMENT

Addressed in: ab875d7113de551fa78231636387f3b042d2892e

SOLVED: The `stake.link` **team** solved this finding in the specified commit by removing the dust-handling block so sub-100-wei withdrawal dust shares stay queued and finalize against real liquidity later, stating:

> Leave sub-100-wei withdrawal dust shares queued so their value finalizes against real liquidity later rather than being discarded.

Addendum: After commit `ab875d7`, a full finalization at a non-integer share price could leave a remainder of shares whose value converts back to zero shares. This remainder stayed counted in `getTotalQueuedWithdrawals()`, so while it was the only queued amount and the Priority Pool queue was empty, Priority Pool deposits would revert with `Transfer amount too small`. The `stake.link` **team** fixed this in commit `142bb97` by excluding the untransferable remainder from the reported queued total while keeping it queued, so it finalizes together with the next queued withdrawal.

HAL-017 SOLVED

Operator reward withdrawal denial of service due to unguarded subtraction in withdrawOperatorRewards

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `OperatorVCS.withdrawOperatorRewards()` function lets a registered vault withdraw accrued operator rewards, paid in the staking pool liquid staking token (stLINK). It caps the withdrawal at the contract's current stLINK balance and then decrements the `unclaimedOperatorRewards` accounting variable by that same balance-capped amount.

However, it decrements `unclaimedOperatorRewards` by the balance-capped amount rather than capping the decrement at `unclaimedOperatorRewards` itself. The contract's stLINK balance can drift above `unclaimedOperatorRewards` because stLINK is a rebasing token: the balance held by `OperatorVCS` grows with positive rebases, while `unclaimedOperatorRewards` was recorded in nominal token terms by `updateDeposits()`, so the share-to-stake conversion can deliver slightly more stLINK than the recorded increment.

When a vault requests an amount at or above the stLINK balance, `amountToWithdraw` is set to the full balance, which exceeds `unclaimedOperatorRewards`, and the decrement underflows and reverts under Solidity 0.8 checked arithmetic. This does not block every withdrawal: a request for an amount at or below `unclaimedOperatorRewards` still succeeds. It blocks any withdrawal whose balance-capped amount exceeds `unclaimedOperatorRewards`, which includes the common case of a vault withdrawing its full available balance, until the next `updateDeposits()` call raises `unclaimedOperatorRewards` back above the contract's stLINK balance.

RECOMMENDATION

Cap `amountToWithdraw` at the minimum of the contract's stLINK balance and `unclaimedOperatorRewards` so the decrement of `unclaimedOperatorRewards` can never underflow.

REMEDIATION COMMENT

Addressed in: fabbc3712e801030d8dca19e3df869b5a00784ed

SOLVED: The `stake.link` **team** solved this finding in the specified commit by capping the `unclaimedOperatorRewards` decrement at `min(balance, unclaimedOperatorRewards)`, stating:

> Cap amountToWithdraw at min(balance, unclaimedOperatorRewards) so the decrement can never underflow.

HAL-018 SOLVED

Permanent reward stranding due to round-down in _updateRewardPerToken

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:L/Y:N

DESCRIPTION

The `RewardsPool._updateRewardPerToken()` function maintains the per-share reward accumulator that drives proportional reward distribution to stLINK stakers. It is called by `RewardsPool.distributeRewards()`, which first adds the full newly deposited amount to the `totalRewards` accounting variable and then advances `rewardPerToken` by `(_reward * 1e18) / totalStaked`.

However, the accumulator update uses integer division that rounds down, and `RewardsPool.withdrawableRewards()` again rounds down when it computes `staked * (rewardPerToken - userRewardPerTokenPaid) / 1e18`. As a result, the sum of all users' claimable rewards is strictly less than the `totalRewards` value that was credited at distribution time. The truncation lost when advancing the accumulator is bounded by less than `totalStaked / 1e18` reward-wei per distribution, plus the per-user truncation incurred on each claim. On the next call to `distributeRewards()`, `toDistribute = token.balanceOf(address(this)) - totalRewards` subtracts the stranded amount back out, so it is never re-distributed, and neither `RewardsPool` nor `RewardsPoolWSD` exposes any sweep or rescue function.

A residual reward balance accumulates permanently in the pool and can never be withdrawn by stakers or recovered by the protocol. The rounding direction keeps the pool solvent and never over-pays, so the impact is stranded value rather than theft. The `RewardsPoolWSD` variant exhibits the same behavior in wrapped-token units.

RECOMMENDATION

Carry the remainder of the division forward in `_updateRewardPerToken()` so that the truncated portion of each distribution is included in a later distribution rather than being permanently stranded. Alternatively, add a privileged sweep limited to the residual that can be shown to be unclaimable, without altering accounting in any way that could make the pool insolvent. Note that `RewardsPool` does not currently expose any ownership or admin role, so this alternative would require introducing one.

REMEDIATION COMMENT

Addressed in: ec62bd5512b8b9766da3751e8a7abebc7b21af0a

SOLVED: The `stake.link` **team** solved this finding in the specified commit by carrying the division remainder forward in `_updateRewardPerToken` so truncated rewards enter a later distribution, stating:

> Carry the division remainder forward in _updateRewardPerToken so truncated rewards enter a later distribution.

HAL-019 SOLVED

Deposit update stalled by permissionless vault deployment during an in-progress batch

Low · 2.5 AO:A/AC:L/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `CommunityVCS.updateVaultDeposits()` function processes deposit-accounting updates in batches, advancing `currentVaultIndex` across the `vaults` array in chunks of `vaultsPerBatch` and accumulating each vault's `getTotalDeposits()` into `totalVaultDepositsAccum`. A batch is considered in progress while `currentVaultIndex` is non-zero. The `CommunityVCS.updateDeposits()` function, callable only by the staking pool, consumes the accumulated total but only once the batch is fully processed, reverting with `DepositUpdateNotReady` whenever `currentVaultIndex < vaults.length`.

However, the `CommunityVCS.performUpkeep()` function is permissionless, having no access control and no `notDuringDepositUpdate` guard, and it deploys `vaultDeploymentAmount` new vaults whenever `(vaults.length - globalVaultState.depositIndex) < vaultDeploymentThreshold`, a condition that is public on-chain state. Because newly deployed vaults are appended to the `vaults` array, any account can call `performUpkeep()` while a batch is in progress and grow `vaults.length` beyond the `currentVaultIndex` the deposit updater has already reached.

After the deposit updater has advanced `currentVaultIndex` up to the previous `vaults.length`, an injected `performUpkeep()` call increases `vaults.length`, so `updateDeposits()` reverts with `DepositUpdateNotReady` and the deposit updater must run additional `updateVaultDeposits()` batches to cover the freshly added vaults before the reward update can complete. The newly deployed vaults are empty, so accounting remains correct, but reward and fee distribution is delayed and the deposit updater's gas is wasted. A single `performUpkeep()` call pushes the spare-vault count above `vaultDeploymentThreshold`, after which the call reverts with `VaultsAboveThreshold`, so the stall is effectively a one-shot delay per batch rather than arbitrarily repeatable. The `addVaults()` function, restricted to the owner, also lacks the in-progress guard and can stall a batch the same way.

RECOMMENDATION

Apply the `notDuringDepositUpdate` guard to `performUpkeep()` and `addVaults()` so that vaults cannot be appended while a deposit-update batch is in progress. Alternatively, snapshot the vault count at the start of a batch and process the update against that snapshot so that vaults added mid-batch do not move the completion target.

REMEDIATION COMMENT

Addressed in: bd0f6fb545ad016cd9736bd1ae849022b9c9df13

SOLVED: The `stake.link` **team** solved this finding in the specified commit by applying the in-progress-batch guard to `performUpkeep` and `addVaults` so vaults cannot be appended mid deposit-update, stating:

> Apply the in-progress-batch guard to performUpkeep / addVaults so vaults cannot be appended mid deposit-update.

HAL-020 RISK ACCEPTED

Permanent loss of administrative control due to single-step ownership transfer

Low · 2.0 AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:N/D:N/Y:N

DESCRIPTION

The `PriorityPool`, `WithdrawalPool`, `StakingPool` (through its `StakingRewardsPool` base), `FundFlowController`, and `SDLPool` (through its `RewardsPoolController` base) contracts all inherit OpenZeppelin's `OwnableUpgradeable`, which exposes a single-step `transferOwnership(address)` that moves ownership to the supplied address immediately, with no acceptance step from the receiving account.

The owner across these contracts holds critical privileges. In `StakingPool` the owner controls `addStrategy()`, `removeStrategy()`, `reorderStrategies()`, `addFee()`, `updateFee()`, `setUnusedDepositLimit()`, `setPriorityPool()`, `setRebaseController()`, and the UUPS upgrade authority in `_authorizeUpgrade()`. In `PriorityPool` the owner controls `setQueueDepositParams()`, `setAllowInstantWithdrawals()`, `setDistributionOracle()`, `setRebaseController()`, `setWithdrawalPool()`, `setQueueBypassController()`, and `_authorizeUpgrade()`, while pool status is governed by `setPoolStatus()`, which is callable by either the owner or the rebase controller. In `WithdrawalPool` the owner controls `forceWithdraw()`, `setMinWithdrawalAmount()`, `setMinTimeBetweenWithdrawals()`, and `_authorizeUpgrade()`. In `FundFlowController` the owner controls `setNonLINKRewardReceiver()`, `delegateVaults()`, and `_authorizeUpgrade()`. In `SDLPool` the owner controls `setBoostController()`, `setBaseURI()`, token management in `addToken()` and `removeToken()`, and `_authorizeUpgrade()`.

However, because the transfer is single-step, if `transferOwnership()` is called with a mistyped or otherwise inaccessible address, ownership moves to that wrong address immediately and irrevocably. There is no pending-owner state and no way for the intended owner to reclaim control.

The result is that a single input error in an ownership transfer permanently bricks all owner-only administration of the affected contract and, because these are UUPS proxies, permanently disables the upgrade path as well, leaving the protocol unable to manage strategies, fees, pool parameters, vault fund flow, reward tokens, or future upgrades.

RECOMMENDATION

Use OpenZeppelin's `Ownable2StepUpgradeable` instead of `OwnableUpgradeable` across the system, so that an ownership transfer requires the new owner to explicitly call `acceptOwnership()` before it takes effect. This prevents ownership from being assigned to an inaccessible address by accident.

REMEDIATION COMMENT

RISK ACCEPTED: The `stake.link` **team** made a business decision to accept the risk of this finding and not alter the contracts, stating:

> Correct, but the recommended Ownable2StepUpgradeable swap is not storage-safe on the live proxies: it inserts _pendingOwner into the linearized chain exactly where each contract's child variables live, shifting every child slot by 50 slots. Adopting it needs a coordinated storage-migration upgrade (same class as HAL-12). Mitigated operationally: ownership is a multisig and transfers are simulated and verified before execution.

HAL-021 ACKNOWLEDGED

Keeper transaction griefing due to exact-match vault deposit ordering check

Informational · 1.7 AO:A/AC:M/AX:L/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `VaultDepositController._depositToVaults()` function deposits queued tokens from the staking pool into the strategy's vaults in a caller-specified order. It is reached when `VaultControllerStrategy.deposit()` (restricted to the staking pool) forwards the call through a delegatecall. To preserve a consistent fill order across calls, the function requires that, when the supplied `_vaultIds` array is non-empty, its first element equals `globalVaultState.groupDepositIndex`, reverting with `InvalidVaultIds` otherwise. As vaults are filled inside the loop, `groupDepositIndex` is advanced to the last processed vault index and written back to storage.

The deposit entry points that ultimately drive this routine, namely `PriorityPool.depositQueuedTokens()` and `PriorityPool.performUpkeep()`, are permissionless and accept the vault ordering data as caller-supplied calldata. A keeper or user that wants to move queued or unused LINK into the staking pool must first read the current ordering data off-chain through `FundFlowController.getDepositData()` and submit a transaction whose first vault id matches the current `groupDepositIndex`.

However, the ordering check is an exact-match comparison against mutable state that any account can advance. Any deposit that lands first and fills the current vault enough to move `groupDepositIndex` forward invalidates the calldata of an in-flight transaction, causing it to revert with `InvalidVaultIds`. A malicious account can deliberately front-run a pending keeper deposit by calling the permissionless `depositQueuedTokens()` with tokens already queued by other users or unused tokens sitting in the pool, so no capital of its own is required, advancing the index and forcing the keeper transaction to revert. The keeper must then re-fetch ordering data and retry, and the front-run can be repeated, so queued-token deposits are delayed and keeper gas is wasted on failed attempts. The same fragility also affects two honest callers acting at the same time: whichever transaction is mined second reverts and must be retried with refreshed data.

No funds are lost and deposits still progress, since the front-running deposit itself moves tokens into vaults productively. The harm is wasted keeper gas, repeated failed transactions, and delayed movement of queued LINK into the staking pool. The vector is low-cost and repeatable because the entry points are permissionless and the ordering check requires an exact match.

RECOMMENDATION

Relax the ordering check in `VaultDepositController._depositToVaults()` so that it accepts a first vault id greater than or equal to the current `groupDepositIndex` and scans forward from

`groupDepositIndex`, skipping already-processed vaults, so that slightly stale ordering data succeeds instead of reverting.
Alternatively, compute the vault deposit order fully on-chain starting from `groupDepositIndex` so that no caller-supplied vault id list is needed and the race is removed entirely. As an additional hardening step, restrict `PriorityPool.depositQueuedTokens()` to a trusted keeper or owner role so that fewer accounts can advance the index adversarially.

REMEDIATION COMMENT

ACKNOWLEDGED: The `stake.link` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> Griefing only: no funds lost, deposits still progress, and the front-running deposit itself moves tokens productively. The report's remedy is a non-trivial rewrite of the core deposit-routing loop on a hot live path where the exact-match check is load-bearing; the regression risk outweighs avoiding occasional keeper retries. The entry point can later be restricted to a trusted keeper role instead.

HAL-022 SOLVED

Opaque panics due to missing bounds and array-length validation

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

Several functions across the contracts in scope index into arrays or read parallel calldata arrays without first validating that the index is within bounds or that the arrays have matching lengths. The intended behavior in each case is a descriptive named revert, and most of these functions are reached with caller-supplied calldata.

However, multiple sites produce a low-level array-out-of-bounds or arithmetic panic instead. The `StakingPool._depositLiquidity()` and `StakingPool._withdrawLiquidity()` functions guard per-strategy data with a non-empty check rather than an index-bound check, so a data array that is non-empty but shorter than the strategy list reverts with a panic mid-loop, reachable through the permissionless deposit and upkeep entry points. The `WithdrawalPool.forceWithdraw()` function never checks that its two calldata arrays have equal length and reads the previous batch at the batch id minus one without confirming the batch id is non-zero, so a zero batch id underflows and the access panics.

The `VaultDepositController.withdraw()` function reads the vault ids array at index zero with no empty-array guard, so an empty list panics rather than producing the named `InvalidVaultIds` error. The `PriorityPool.getLSDTokensBatch()` function reads the distribution-share-amounts array at the account index without checking the two arrays have equal length. The `VaultControllerStrategy.setWithdrawalIndexes()` function loops over the configured number of vault groups reading the supplied array at each position without confirming it is long enough. The `CommunityVCS.claimRewards()` function indexes the vaults array with caller-supplied indexes with no bounds check. The `RebaseController.performUpkeep()` function decodes a strategy index from caller-supplied data and indexes the strategies array without confirming it is in range.

In all cases no funds are at risk and state-changing paths still revert correctly. The consequence is a confusing, non-descriptive failure mode that complicates integration, keeper tooling, and incident response, and through the permissionless deposit and upkeep paths it can cause queued-token deposits to fail and waste keeper gas.

RECOMMENDATION

Validate array lengths and indexes before indexing in each function, reverting with a clear named error otherwise. For per-strategy data routing in `StakingPool._depositLiquidity()` and `StakingPool._withdrawLiquidity()`, fall back to empty bytes when the index is beyond the provided array length rather than only checking that the array is non-empty, or require at function entry that the data array is either empty or exactly the strategy-list length. Additionally, add the

matching empty-array, equal-length, non-zero, and in-range checks to `WithdrawalPool.forceWithdraw()`, `VaultDepositController.withdraw()`, `PriorityPool.getLSDTokensBatch()`, `VaultControllerStrategy.setWithdrawalIndexes()`, `CommunityVCS.claimRewards()`, and `RebaseController.performUpkeep()`, mirroring the guards already present in the sibling deposit and withdraw functions, and apply the strategy-side changes to both the `CommunityVCS` and `OperatorVCS` base strategies.

REMEDIATION COMMENT

Addressed in: d8c4d4d1e5946f8752e47f881c9a04af7734c80b

SOLVED: The `stake.link` **team** solved this finding in the specified commit by validating array lengths and indexes before use across all the flagged external entry points and replacing the opaque panics with named errors, stating:

> Added bounds and array-length validation across the flagged functions, replacing opaque panics with named reverts.

HAL-023 ACKNOWLEDGED

Gas griefing through permissionless updateOperatorVaultGroupAccounting with caller-controlled array

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `FundFlowController.updateOperatorVaultGroupAccounting()` function recomputes deposit room and unbonded totals for a caller-supplied list of vault groups. For each entry it calls `_getTotalDepositRoom()`, which makes two external calls per vault, and conditionally `_getTotalUnbonded()`, which makes up to three external calls per vault.

However, the function does not bound or deduplicate the `_vaultGroups` array. Any account can call it with a long array containing repeated group indices, forcing the function to iterate the vaults of those groups many times over and consume a large amount of gas in a single transaction.

The practical impact is gas griefing in which the caller burns its own gas to force repeated recomputation of values that are objective on-chain reads. No accounting can be corrupted, since the values are recomputed from live vault state on each call. The cost is confined to the griefing caller's own transaction. The internal single-group call made by `OperatorVCS.queueVaultRemoval()` builds its own length-one array and is unaffected by any oversized array an attacker passes, because each invocation iterates only the groups in its own argument.

RECOMMENDATION

Bound and deduplicate the `_vaultGroups` array so the number of vault groups processed in a single call cannot exceed the configured number of vault groups. Do not add a trusted-caller restriction, because `OperatorVCS.queueVaultRemoval()` invokes this function internally on the permissionless removal path and gating it would break operator-vault removal. If a caller restriction is nonetheless desired, the `OperatorVCS` strategy must remain an authorized caller.

REMEDIATION COMMENT

ACKNOWLEDGED: The `stake.link` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> Self-griefing only: the cost is confined to the caller's own transaction, and duplicate indices rewrite the same recomputed values idempotently. The function must stay permissionless because queueVaultRemoval calls it internally on the permissionless removal path.

HAL-024 SOLVED

Withdrawal denial of service due to unguarded subtraction in getMinDeposits after slashing

Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `VaultControllerStrategy.getMinDeposits()` function reports the minimum amount of tokens that must remain in the strategy and therefore cannot be withdrawn. It is the shared base used by both `CommunityVCS` and `OperatorVCS`, and the staking pool relies on its return value to decide how much can be withdrawn. While a claim period is active, it computes the floor as `totalDeposits` minus `totalUnbonded`, where `totalDeposits` is the strategy's tracked balance and `totalUnbonded` is the amount currently available to withdraw from the unbonded vault group.

However, this subtraction has no lower-bound guard. The `totalDeposits` value is reduced by `updateDeposits()` whenever a slashing event produces a negative deposit change, but `totalUnbonded` is only refreshed by `FundFlowController.updateVaultGroups()` and `FundFlowController.updateOperatorVaultGroupAccounting()`. Between a slashing event and the next vault-group accounting update, a stale `totalUnbonded` can exceed the reduced `totalDeposits`, so the subtraction underflows and reverts under Solidity 0.8 checked arithmetic.

The revert propagates through `StakingPool.canWithdraw()`, which sums every strategy's `getMinDeposits()`, and into the withdrawal-capacity check in the priority pool. Because the sum spans all strategies, a single underflowing strategy blocks all withdrawals from the pool, not just withdrawals routed to the affected strategy. Any user attempting a withdrawal triggers the revert, so the condition is not specific to a privileged or malicious caller. The same logic is present in both the `CommunityVCS` and `OperatorVCS` base strategies. The condition self-heals once anyone calls the permissionless `updateOperatorVaultGroupAccounting()` or `updateVaultGroups()`, but during that window every pool withdrawal is unavailable.

RECOMMENDATION

Replace the raw subtraction in `getMinDeposits()` with a saturating subtraction that returns zero whenever `totalUnbonded` is greater than or equal to `totalDeposits`, so the function can never revert on underflow. Returning zero is correct because all deposits are effectively withdrawable when the unbonded amount meets or exceeds the reduced deposit total. Apply the same change to both the `CommunityVCS` and `OperatorVCS` base strategies.

Additionally, re-sync `totalUnbonded` against the reduced `totalDeposits` as part of the slashing-response and accounting flow, by triggering `updateOperatorVaultGroupAccounting()` or `updateVaultGroups()`, so that the reported withdrawal floor stays consistent after a slash.

REMEDIATION COMMENT

Addressed in: 58b8d7e418a31045c60265420de4580abcaa0de8

SOLVED: The `stake.link` **team** solved this finding in the specified commit by replacing the raw subtraction in `getMinDeposits()` with a saturating subtraction that returns zero when `totalUnbonded` meets or exceeds `totalDeposits`, stating:

> Saturating subtraction in getMinDeposits so a stale totalUnbonded after a slash cannot block all withdrawals.

HAL-025 ACKNOWLEDGED

Unbounded iteration in getLockIdsByOwner can revert as the global lock counter grows

• Informational · 1.7 AO:A/AC:L/AX:M/R:N/S:U/C:N/A:L/I:N/D:N/Y:N

DESCRIPTION

The `SDLPool.getLockIdsByOwner()` function returns the list of lock IDs owned by an account. It iterates from `i = 1` up to `lastLockId`, checks `lockOwners[i] == _owner`, collects matches into a fixed-size array sized by `balanceOf(_owner)`, and concludes with `assert(lockIdsFound == lockCount)`. It is the canonical helper for off-chain components, front-ends, and any on-chain integrator that needs to enumerate an owner's locks.

However, the loop is bounded by `lastLockId`, a counter incremented in `SDLPool._createLock()` for every lock ever created across all accounts and never decremented when a lock is withdrawn and deleted in `SDLPool.withdraw()`. The iteration length therefore grows without bound over the protocol's lifetime, independent of how many locks the queried account actually holds. The early break only helps when the queried account owns locks at low indices: an account that owns no matching locks, or whose locks sit near the end of the id space, forces a scan of the entire range. New locks are created by any account through `SDLPool.onTokenTransfer()` by staking SDL, including with a zero `lockingDuration`.

As `lastLockId` grows large, the cost of `getLockIdsByOwner()` grows linearly and can exceed the node gas cap for an `eth_call` or the block gas limit for any on-chain caller, causing the call to revert. Any consumer that depends on enumerating an owner's locks is then unable to do so until it switches to a different enumeration method. An attacker can deliberately accelerate the growth by cheaply mass-creating minimal zero-duration locks, hastening the point at which the function becomes unusable. There is no fund loss; the impact is denial of a read helper.

RECOMMENDATION

Avoid scanning the entire global lock-id space. Maintain a per-owner enumerable collection of lock IDs, for example a per-account array kept in sync on lock creation, transfer, withdrawal, and deletion, or an OpenZeppelin `EnumerableSet`, so that enumeration cost scales with the owner's own lock count rather than with the global `lastLockId` counter.

REMEDIATION COMMENT

ACKNOWLEDGED: The `stake.link` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> external view helper; no fund or state risk. The recommended per-owner enumerable index is a sizeable, upgrade-sensitive state addition to the live SDLPool that must stay in sync on every

> create, transfer, withdrawal, and merge, disproportionate to a read-helper DoS. The one on-chain consumer queries a single known account and benefits from the early break.

HAL-026 ACKNOWLEDGED

Core ERC20 invariant broken because the dust guard hides small balances from balanceOf

• Informational · 0.8 AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:L/D:N/Y:N

DESCRIPTION

The `StakingRewardsPool` contract implements the liquid staking token as a rebasing ERC20 in which `totalSupply()` returns the raw total staked value, while `balanceOf()` converts an account's shares to a token amount and then applies a dust guard that returns zero whenever the result is below 100 wei.

However, this dust guard makes `balanceOf()` under-report any account whose token-equivalent balance is between 1 and 99 wei, while `totalSupply()` continues to include those balances. As a result, the core ERC20 invariant that `totalSupply()` equals the sum of all `balanceOf()` values is violated whenever any account holds a sub-100-wei dust balance. Separately, these dust positions can also become non-transferable in the specific case where the share price has risen enough that converting the token amount to shares rounds down to zero, since `_transfer()` and `_burn()` revert with "Transfer amount too small" and "Burn amount too small" when the derived share amount is zero. When the share price is near one token, a sub-100-wei balance still converts to a non-zero share amount and remains transferable, so the transfer and burn limitation applies only when the amount-to-shares conversion rounds to zero, not to every dust balance.

The consequence is that integrators that rely on the standard ERC20 invariant, such as on-chain solvency or collateral checks that compare `totalSupply()` against summed balances, can observe a discrepancy. The magnitude of the discrepancy is bounded by the number of dust-holding accounts times 99 wei, so the practical financial impact is negligible, but the standard invariant does not hold.

RECOMMENDATION

Apply the dust guard consistently so that `totalSupply()` and `balanceOf()` agree. Alternatively, remove the dust guard from `balanceOf()` and prevent dust accumulation through a minimum deposit amount instead. If the dust guard is retained, document explicitly in NatSpec that `totalSupply()` is not equal to the sum of `balanceOf()` for accounts holding sub-threshold dust, so that integrators do not rely on the invariant.

REMEDIATION COMMENT

ACKNOWLEDGED: The `stake.link` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> The dust guard is intentional and paired with the transfer/burn "amount too small" guards; a sub-threshold balance rounds to zero shares and cannot be moved anyway. Removing it would make

> balanceOf report unmovable 1-99 wei on a widely integrated token, and excluding dust from totalSupply is infeasible. Per the report's fallback, we keep the guard and document in NatSpec that the sum invariant does not hold for sub-threshold dust.

HAL-027 SOLVED

Transfer event reports nominal amount instead of the actual transferred value

Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

The `StakingRewardsPool._transfer()` function moves a liquid staking token balance by converting the requested token amount to shares with `getSharesByStake(_amount)` and transferring those shares. Because that conversion uses integer division, and `balanceOf()` performs a second integer division when converting shares back to a token amount, the token value actually credited to the recipient can differ from the requested `_amount`.

However, `_transfer()` emits `Transfer(_sender, _recipient, _amount)` using the original requested amount rather than the actual token value moved. This causes the emitted event to over-report the transferred value by the rounding remainder. Because two floor divisions are involved, the discrepancy is bounded by approximately the token value of one share, which exceeds one wei once the share price rises above one token. The share-based transfer function `_transferShares()` does this correctly, emitting `getStakeByShares(_sharesAmount)`, so the two transfer paths are inconsistent.

The consequence is that off-chain consumers that track balances from `Transfer` events, such as indexers and integrating contracts, can record a value slightly higher than what was actually credited on chain. The financial impact is negligible, but the event does not faithfully reflect the on-chain state change.

RECOMMENDATION

Emit the actual transferred token value in `_transfer()` by computing `getStakeByShares(sharesToTransfer)` and using it in the `Transfer` event, consistent with the behavior of `_transferShares()`.

REMEDIATION COMMENT

Addressed in: 29e3a3893460b89fb960cad9dedc5977b8cb6a5f

SOLVED: The `stake.link` **team** solved this finding in the specified commit by emitting the actual transferred value (`getStakeByShares(sharesToTransfer)`) in `StakingRewardsPool._transfer()`, stating:

> Emit the actual transferred token value in _transfer rather than the nominal requested amount.

HAL-028 ACKNOWLEDGED

Checks-effects-interactions violation in removeVault during vault exit

• Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

The `OperatorVCS.removeVault()` function withdraws a queued vault's funds and de-registers it from the strategy. It first calls `_updateStrategyRewards()`, then makes the external call `IOperatorVault(vault).exitVault()`, and only afterward clears the vault's authorization entry with `delete vaultMapping[vault]`.

However, this orders the external interaction before the state effect, violating the checks-effects-interactions pattern. While `exitVault()` executes, the vault is still registered in `vaultMapping`, so it remains an authorized caller of `OperatorVCS.withdrawOperatorRewards()`. That function authorizes callers by `vaultMapping[msg.sender]` and caps withdrawals at the contract's full stLINK balance `lsdToken.balanceOf(address(this))` rather than at the calling vault's own accrued rewards. A re-entrant call from inside `exitVault()` could therefore withdraw stLINK belonging to other operators before the vault is de-registered.

Operator vaults are protocol-deployed ERC1967 proxies pointing at an owner-controlled implementation, so reaching this path requires a vault implementation that re-enters during `exitVault()`, which is not the case for the audited implementation. The issue is a latent reentrancy surface that would become exploitable if a vault implementation were ever upgraded to untrusted or buggy code, at which point a removed vault could drain the shared stLINK balance and leave the remaining operators unable to withdraw their rewards.

RECOMMENDATION

Reorder `removeVault()` so that all state effects, including `delete vaultMapping[vault]` and the removal-queue updates, are applied before the external `exitVault()` call. Additionally, bound `withdrawOperatorRewards()` to each vault's individually tracked reward allocation rather than the contract's global stLINK balance, and consider applying a reentrancy guard to `removeVault()` and `withdrawOperatorRewards()`.

REMEDIATION COMMENT

ACKNOWLEDGED: The `stake.link` **team** made a business decision to acknowledge this finding and not alter the contracts, stating:

> Correct, but the obvious fixes break a legitimate flow: exitVault re-enters withdrawOperatorRewards so a departing vault can claim its own rewards, so de-registering first or adding a reentrancy guard makes that legitimate callback fail (confirmed by a reverting test).

> The robust fix (per-vault reward accounting) is a feature-sized change with regression risk on a live contract. Latent: operator vaults run a protocol-controlled implementation that does not re-enter during exitVault.

HAL-029 SOLVED

Loss of revert context due to discarded delegatecall return data in deposit and withdraw

Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

The `VaultControllerStrategy.deposit()` and `VaultControllerStrategy.withdraw()` functions route deposits and withdrawals to the vault deposit controller through a `delegatecall`, capturing only a success boolean. On failure they revert with the generic `DepositFailed` or `WithdrawalFailed` custom error. This shared base is used by both `CommunityVCS` and `OperatorVCS`.

However, both functions discard the return data of the `delegatecall`, so the original revert reason is lost. Distinct downstream failure modes such as `InvalidVaultIds` (incorrect vault ordering), `InsufficientTokensUnbonded` (claim period not active or amount exceeds unbonded), and vault-level panics are all collapsed into the same opaque `DepositFailed` or `WithdrawalFailed` error.

The functions still revert correctly on failure, so there is no direct security impact. The consequence is reduced diagnostic fidelity: higher-level contracts and off-chain monitoring cannot distinguish a transient timing error from a configuration error or an internal vault failure, which complicates incident response and recovery.

RECOMMENDATION

Bubble up the original revert data from the delegatecall when the call fails, falling back to the generic `DepositFailed` or `WithdrawalFailed` error only when no return data is present, and apply the change to both `deposit()` and `withdraw()` in both base strategies.

REMEDIATION COMMENT

Addressed in: 018fb64e32563904fa8f83754354b3d6fc0ba75a

SOLVED: The `stake.link` **team** solved this finding in the specified commit by routing the delegatecalls through `AddressUpgradeable.functionDelegateCall`, which bubbles up the original revert reason, stating:

> Bubble up the original delegatecall revert data in deposit / withdraw.

HAL-030 SOLVED

Missing events on state-changing administrative functions

Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

Throughout the contracts in scope, there are several instances where administrative functions change contract state by modifying core state variables without them being reflected in event emissions.

The absence of events may hamper effective state tracking in off-chain monitoring systems.

Instances of this issue can be found in:

- `StakingPool.removeStrategy()`
- `StakingPool.reorderStrategies()`
- `StakingPool.updateFee()`
- `StakingPool.setUnusedDepositLimit()`
- `StakingPool.setPriorityPool()`
- `StakingPool.setRebaseController()`
- `WithdrawalPool.updateWithdrawalBatchIdCutoff()`
- `PriorityPool.setAllowInstantWithdrawals()`
- `PriorityPool.setDistributionOracle()`
- `PriorityPool.setRebaseController()`
- `PriorityPool.setWithdrawalPool()`
- `PriorityPool.setQueueBypassController()`

RECOMMENDATION

Emit events for all state changes that occur as a result of administrative functions to facilitate off-chain monitoring of the system.

REMEDIATION COMMENT

Addressed in: 5d6513a7f4597345b6a2672cad694115cef2db82

SOLVED: The `stake.link` **team** solved this finding in the specified commit by adding events to every listed administrative function, stating:

> Added events to the flagged state-changing administrative setters.

HAL-031 SOLVED

Missing zero-address validation on critical immutable addresses

Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

The initializers and setters across the contracts in scope assign critical, effectively immutable address parameters to state variables. For these addresses a zero value is never a valid configuration, and on a proxy it would brick the affected flow while remaining expensive or impossible to correct.

However, several of these assignments lack a zero-address check on parameters that the protocol always requires to be set. The `StakingRewardsPool.__StakingRewardsPool_init()` function assigns `_token` without a zero-address check, `StakingPool.initialize()` assigns `_token` without a zero-address check, `PriorityPool.initialize()` assigns `_token`, `_stakingPool`, and `_sdlPool` without zero-address checks and does not verify that `_queueDepositMin` is less than or equal to `_queueDepositMax`, `PriorityPool.setQueueDepositParams()` does not verify that `_queueDepositMin` is less than or equal to `_queueDepositMax`, and `WithdrawalPool.initialize()` assigns `_token`, `_lst`, and `_priorityPool` without zero-address checks.

This scope is intentionally limited to addresses and parameters with a clear invariant. Other setter parameters are excluded on purpose: limits such as `unusedDepositLimit` have no natural bound and accept zero as a meaningful value, and some address setters may legitimately accept the zero address as a disable state, so blanket non-zero or range validation is not appropriate for them.

RECOMMENDATION

Add zero-address validation for the critical immutable address parameters listed above, and enforce that `queueDepositMin` is less than or equal to `queueDepositMax` where those parameters are set. Do not apply blanket non-zero or range validation to parameters that do not have a justified invariant.

REMEDIATION COMMENT

Addressed in: 656069b01d9799d405662397bd2dacd749cc98f0

SOLVED: The `stake.link` **team** solved this finding in the specified commit by adding zero-address checks on the critical immutable address parameters and enforcing `queueDepositMin <= queueDepositMax`, stating:

> Added zero-address and parameter validation on the flagged initializers and setters.

HAL-032 ACKNOWLEDGED

Withdrawal can transfer more than the requested amount when avoiding a sub-minimum vault remainder

Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

The `VaultDepositController.withdraw()` function, executed against a strategy's storage through a delegatecall from `VaultControllerStrategy.withdraw()` (restricted to the staking pool), is intended to withdraw exactly the requested `_amount` of LINK from the strategy's unbonded vaults and transfer it to the staking pool. The same logic backs both the OperatorVCS and CommunityVCS base strategies.

However, to avoid leaving a vault below the Chainlink per-vault minimum stake, the middle branch of the withdrawal loop withdraws the vault's full principal and breaks when `0 < deposits - toWithdraw < minDeposits`, decrementing the running unbonded amount by the full `deposits` even though only `toWithdraw` was still required. The resulting total withdrawn can therefore exceed the requested `_amount` by up to `minDeposits - 1`, and the surplus is transferred to the staking pool. This is not a loss: the surplus lands in the staking pool as undeployed balance and is redeposited on the next cycle, and the accounting identity that ties the pool's tracked stake to the sum of strategy deposits plus undeployed balance is preserved. The behavior is a deliberate trade-off to respect the per-vault minimum, with the only side effects being a transient over-reduction of the unbonded total for the current claim period and a less descriptive flow.

RECOMMENDATION

If exact-amount withdrawals are desired, skip the vault that cannot be partially drained and continue to the next eligible vault rather than draining its full principal, or cap the amount credited and transferred at `_amount` while reconciling the unbonded and deposit accounting. Otherwise, document that the withdrawal may return up to one minimum-deposit unit more than requested by design and that the surplus is reconciled as undeployed pool balance.

REMEDIATION COMMENT

ACKNOWLEDGED: The `stake.link` **team** made a business decision to acknowledge this finding and not alter the contracts, documented as by-design per the recommendation's accepted fallback, stating:

> Intentional trade-off to respect Chainlink's per-vault minimum stake: a withdrawal that would strand a sub-minimum remainder fully drains that vault instead. The surplus (bounded by minDeposits - 1) lands in the pool as undeployed balance and is redeposited next cycle; the accounting identity is preserved. Documented rather than adding complexity to the withdrawal loop.

HAL-033 SOLVED

Non-empty placeholder data passed to token-transfer callbacks

Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

The `StakingPool._updateStrategyRewards()` function transfers fee shares to receivers with a `transferAndCallFrom()` call that passes the string literal `"0x"` as the data argument, and the `RewardsPoolWSD.distributeRewards()` function wraps the pool balance with a `transferAndCall()` call that passes the same `"0x"` literal. When a receiver is a contract, `ERC677Upgradeable.contractFallback()` forwards this data to the receiver's `onTokenTransfer()` callback.

However, the string literal `"0x"` is not empty calldata. It encodes the two ASCII bytes `0x3078`, so every callback is invoked with a two-byte payload rather than the empty bytes that the call clearly intends. A receiver or wrapper that interprets the data argument, for example expecting it to be empty or to decode into a specific structure, can misbehave or revert when it receives the unexpected `0x3078` bytes.

RECOMMENDATION

Pass empty calldata by using an empty string instead of the string literal `"0x"` in the `transferAndCallFrom()` calls within `StakingPool._updateStrategyRewards()` and in the `transferAndCall()` call within `RewardsPoolWSD.distributeRewards()`, unless the receiver or wrapper specifically requires that payload.

REMEDIATION COMMENT

Addressed in: 4210a35e08a51ae8db6363639be4fa034f6ea9ae

SOLVED: The `stake.link` **team** solved this finding in the specified commit by passing empty bytes (`""`) instead of the `"0x"` string literal to the token-transfer callbacks, stating:

> Pass empty calldata instead of the "Ox" literal to token-transfer callbacks.

HAL-034 SOLVED

Batch token distribution reverts entirely when any reward pool has a zero balance

Informational · AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N

DESCRIPTION

The `RewardsPoolController.distributeToken()` function moves the controller's full balance of a supported token into the token's associated rewards pool and triggers distribution. The `RewardsPoolController.distributeTokens()` function exposes a permissionless batch wrapper that loops over a caller-supplied list of tokens and calls `distributeToken()` for each one, so keepers can settle rewards for several reward tokens in a single transaction.

However, `distributeToken()` requires a strictly positive balance and reverts whenever the controller currently holds no balance of the given token. Because `distributeTokens()` performs no per-token balance check and does not isolate the individual calls, the entire batch reverts if any single token in the supplied list happens to have a zero balance at that moment, even when every other token in the list has a distributable balance.

A routine multi-token distribution can fail because one unrelated supported token is momentarily empty, forcing callers to pre-filter the token list off-chain and reducing keeper reliability. No funds are lost and per-token reward accounting is unaffected.

RECOMMENDATION

In `RewardsPoolController.distributeTokens()`, skip tokens whose controller balance is zero rather than letting `distributeToken()` revert the whole batch, or change `distributeToken()` to return early without reverting on a zero balance so that batch distribution tolerates empty token balances.

REMEDIATION COMMENT

Addressed in: 274bd7f02186d866dbfdcdc61328da77b627a307

SOLVED: The `stake.link` **team** solved this finding in the specified commit by skipping tokens with a zero controller balance inside the `distributeTokens` batch loop, stating:

> distributeTokens skips tokens with a zero balance instead of reverting the whole batch.

Disclaimer

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.